REQUISITOS FUNCIONALES

#	Requisito funcional	Descripción
1	Registrar equipos	Permitir registrar equipos con código único, nombre y empresa. Cada equipo inicia con cero puntos de tabla.
2	Validar códigos de equipos	Rechazar códigos vacíos o repetidos.
3	Crear encuentros	Permitir crear encuentros de fútbol sala o baloncesto entre dos equipos registrados y diferentes. Inician sin programar y sin resultado.
4	Crear exhibiciones	Permitir crear exhibiciones con un título, sin equipos, marcador ni puntos de tabla. Inician sin programar.
5	Programar eventos	Asignar a encuentros y exhibiciones una fecha válida, una hora entera entre 0 y 23 y una sede.
6	Reprogramar eventos	Permitir modificar la programación de exhibiciones y encuentros que todavía no tengan resultado.
7	Validar disponibilidad de sede	Rechazar la programación si otro evento ocupa la misma sede, fecha y hora. Al reprogramar, excluir al propio evento de la comparación.
8	Validar disponibilidad de equipos	Rechazar la programación de un encuentro si alguno de sus equipos tiene otro encuentro en la misma fecha y hora.
9	Registrar resultados	Permitir registrar resultados únicamente en encuentros programados, con marcadores enteros no negativos.
10	Asignar puntos en fútbol sala	Otorgar 3 puntos al ganador y 0 al perdedor. Si hay empate, otorgar 1 punto a cada equipo.
11	Asignar puntos en baloncesto	Rechazar empates y otorgar 2 puntos al ganador y 1 al perdedor.
12	Finalizar encuentros	Guardar el resultado válido, sumar los puntos correspondientes una sola vez y marcar el encuentro como finalizado.
13	Proteger encuentros finalizados	Impedir que un encuentro finalizado sea reprogramado o reciba otro resultado.
14	Mostrar tabla de posiciones	Presentar los equipos ordenados por puntos de mayor a menor y, en caso de empate, por nombre en orden alfabético.
15	Mostrar calendario	Presentar los eventos con programación, incluidos los encuentros finalizados y las exhibiciones, ordenados por fecha, hora y sede.
16	Cargar datos iniciales	Iniciar con al menos cuatro equipos de dos empresas, cuatro encuentros que incluyan ambos deportes, dos encuentros finalizados y una exhibición programada. Los datos deben mostrar un empate en fútbol sala y un desempate por nombre en la tabla.
17	Mostrar menú de operaciones	Mantener disponible un menú de consola con las operaciones del sistema hasta que el usuario decida salir.
18	Manejar entradas y operaciones inválidas	Mostrar un mensaje claro ante errores, conservar el estado anterior y permitir que el programa continúe funcionando.

Preguntas del análisis

Reglas por deporte y comportamientos compartidos

Regla o comportamiento	Quién lo resuelve	Motivo
Validar el marcador de fútbol sala y permitir empates.	Clase concreta FutbolSala.	Es una regla propia de ese deporte.
Validar el marcador de baloncesto y rechazar empates.	Clase concreta Baloncesto.	Sus resultados tienen condiciones diferentes.
Calcular los puntos según el resultado.	Clases concretas FutbolSala y Baloncesto.	Cada deporte reparte los puntos de manera distinta.
Guardar equipos, marcador, estado y programación de un encuentro.	Clase abstracta Encuentro.	Son datos comunes a todos los encuentros.
Registrar el resultado, sumar puntos y finalizar.	Clase abstracta Encuentro.	Comparte el procedimiento y llama a los métodos de cada deporte mediante polimorfismo.
Programar eventos y consultar su información para el calendario.	Interfaz Programable.	Establece un contrato común para encuentros y exhibiciones.
Registrar resultados y calcular puntos.	Interfaz Puntuable.	Agrupa las operaciones de los encuentros que participan en la tabla.
Detectar conflictos de sede y equipos.	Clase concreta Copa.	Tiene acceso a los eventos necesarios para compararlos.

Estados de los eventos

Evento	Estado inicial	Operación	Estado resultante
Encuentro	Sin programar	Programar con datos válidos y sin conflictos.	Programado
Encuentro	Programado	Reprogramar con datos válidos y sin conflictos.	Programado, con nueva programación

Encuentro	Programado	Registrar un resultado válido.	Finalizado
Encuentro	Finalizado	Intentar reprogramar o registrar otro resultado.	La operación se rechaza y continúa finalizado.
Exhibición	Sin programar	Programar con datos válidos y sin conflictos.	Programada
Exhibición	Programada	Reprogramar con datos válidos y sin conflictos.	Programada, con nueva programación

Clases, responsabilidades e inicialización

Clase o interfaz	**Responsabilidad**
Equipo	Guardar los datos del equipo, acumular sus puntos y permitir su ordenamiento.
Encuentro, abstracta	Reunir los datos y operaciones comunes de los encuentros.
FutbolSala	Validar sus marcadores y aplicar el reparto de puntos de fútbol sala.
Baloncesto	Rechazar empates y aplicar el reparto de puntos de baloncesto.
Exhibicion	Representar un evento que puede programarse, pero no recibe resultados.
Copa	Administrar equipos y eventos, validar conflictos y proporcionar la tabla y el calendario.
Vista	Leer las entradas y mostrar menús, resultados y mensajes.
Controlador	Coordinar las acciones de la vista y el modelo.
Main	Crear el controlador e iniciar el programa.
Programable, interfaz	Definir las operaciones comunes de programación y consulta de eventos.
Puntuable, interfaz	Definir las operaciones para registrar resultados y calcular puntos.

Los objetos reciben sus valores iniciales mediante sus constructores:

- Equipo recibe código, nombre y empresa e inicia con cero puntos.
- Encuentro recibe su identificador y sus dos equipos. Inicia sin programación, con marcadores y fecha en null, hora en -1 y sede en null.

- Los constructores de FutbolSala y Baloncesto llaman a super(...) para que Encuentro inicialice los datos heredados.
- Exhibicion recibe identificador y título e inicia sin programación.
- Copa crea las listas, establece el siguiente identificador en 1 y llama a cargarDatosIniciales(). Esta carga utiliza las operaciones del modelo para que los resultados iniciales también sumen sus puntos.
- Vista inicializa su Scanner, y Controlador crea la vista y la copa.

Separación de las dos interfaces

El calendario y la tabla necesitan comportamientos diferentes. Programable permite trabajar con cualquier evento que pueda tener fecha, hora y sede, incluyendo encuentros y exhibiciones. Puntuable define las operaciones relacionadas con resultados y puntos, que corresponden únicamente a los encuentros.

Si las exhibiciones tuvieran que implementar Puntuable, estarían obligadas a incluir métodos para registrar marcadores y calcular puntos que no tienen sentido para ellas. Separar las interfaces permite que cada clase implemente las operaciones que necesita.

Además, Comparable establece cómo ordenar los equipos y los eventos, mientras que Iterable permite recorrer el calendario.

Incorporación de un tercer deporte

Para agregar otro deporte se debe crear una subclase de Encuentro e implementar validarMarcador(), calcularPuntos() y getDeporte(). Su constructor debe llamar a super(...) para inicializar los datos comunes.

También se debe agregar en Copa una operación para crear encuentros del nuevo deporte y actualizar el menú y el controlador para permitir seleccionarlo.

La tabla de posiciones y el calendario mantienen su funcionamiento si el nuevo deporte utiliza los mismos criterios de clasificación y programación. La tabla recibe los puntos calculados por la nueva subclase, y el calendario trabaja con ella mediante Programable.

TABLA PROPIEDADES

#	Nombre	Tipo	Visibilidad	Clase	Motivo
1	codigo	String	private	Equipo	Identificar al equipo con un código único.
2	nombre	String	private	Equipo	Guardar el nombre del equipo.
3	empresa	String	private	Equipo	Guardar la empresa que representa.
4	puntosTabla	int	private	Equipo	Acumular los puntos de tabla. Inicia en 0.
5	id	int	private	Encuentro	Identificar el encuentro.
6	equipo1	Equipo	private	Encuentro	Guardar el primer equipo participante.
7	equipo2	Equipo	private	Encuentro	Guardar el segundo equipo participante.
8	marcador1	Integer	private	Encuentro	Guardar el marcador del primer equipo. Inicia en null.
9	marcador2	Integer	private	Encuentro	Guardar el marcador del segundo equipo. Inicia en null.
10	estado	String	private	Encuentro	Indicar si está sin programar, programado o finalizado.
11	fecha	LocalDate	private	Encuentro	Guardar la fecha del encuentro. Inicia en null.
12	hora	int	private	Encuentro	Guardar la hora de inicio. Inicia en −1.
13	sede	String	private	Encuentro	Guardar la sede del encuentro. Inicia en null.
14	id	int	private	Exhibicion	Identificar la exhibición.
15	titulo	String	private	Exhibicion	Guardar el título de la exhibición.
16	fecha	LocalDate	private	Exhibicion	Guardar la fecha de la exhibición. Inicia en null.
17	hora	int	private	Exhibicion	Guardar la hora de inicio. Inicia en −1.
18	sede	String	private	Exhibicion	Guardar la sede de la exhibición. Inicia en null.
19	equipos	ArrayList<Equipo>	private	Copa	Almacenar los equipos registrados.
20	eventos	ArrayList<Programable>	private	Copa	Almacenar las referencias de encuentros y exhibiciones.
21	encuentros	ArrayList<Puntuable>	private	Copa	Almacenar las referencias de los encuentros que pueden recibir resultados.
22	siguienteId	int	private	Copa	Generar identificadores únicos para los eventos. Inicia en 1.

23	copa	Copa	private	Controlador	Guardar la copa que el controlador crea en su constructor y consultar sus datos y operaciones.
24	vista	Vista	private	Controlador	Guardar la vista que el controlador crea en su constructor para solicitar datos y presentar información.
25	scanner	Scanner	private	Vista	Leer las entradas del usuario en consola.

TABLA MÉTODOS

#	Nombre	Tipo	Visibilidad	Clase	Parámetros	Motivo	Excepciones
1	getId	int	public	Programable		Obtener el identificador del evento.	
2	getFecha	LocalDate	public	Programable		Obtener la fecha asignada.	
3	getHora	int	public	Programable		Obtener la hora asignada.	
4	getSede	String	public	Programable		Obtener la sede asignada.	
5	estaProgramado	boolean	public	Programable		Indicar si el evento tiene programación.	
6	programar	void	public	Programable	LocalDate fecha, int hora, String sede	Establecer el contrato para programar o reprogramar un evento.	IllegalArgumentException, IllegalStateException.
7	participaEquipo	boolean	public	Programable	Equipo equipo	Consultar si un equipo participa en el evento.	
8	getDescripcion	String	public	Programable		Obtener la información del evento.	
9	compareTo	int	public, default	Programable	Programable otro	Comparar eventos programados por fecha, hora y sede con un solo criterio.	
10	getId	int	public	Puntuable		Obtener el identificador del encuentro.	
11	getDescripcion	String	public	Puntuable		Obtener la descripción del encuentro.	
12	registrarResultado	void	public	Puntuable	int marcador1, int marcador2	Establecer el contrato para registrar resultados y repartir puntos.	IllegalArgumentException, IllegalStateException.
13	calcularPuntos	int[]	public	Puntuable	int marcador1, int marcador2	Obtener los puntos de tabla correspondientes a cada equipo.	IllegalArgumentException
14	Equipo	Constructor	public	Equipo	String codigo, String nombre, String empresa	Inicializar el equipo con cero puntos y validar sus datos.	IllegalArgumentException
15	getCodigo	String	public	Equipo		Consultar el código del equipo.	
16	getNombre	String	public	Equipo		Consultar el nombre del equipo.	
17	getEmpresa	String	public	Equipo		Consultar la empresa que representa.	
18	getPuntosTabla	int	public	Equipo		Consultar los puntos acumulados.	
19	sumarPuntos	void	public	Equipo	int puntos	Sumar una cantidad no negativa de puntos de tabla.	IllegalArgumentException
20	compareTo	int	public	Equipo	Equipo otro	Ordenar por puntos descendentes y, en caso de empate, por nombre ascendente.	
21	Encuentro	Constructor	protected	Encuentro	int id, Equipo equipo1, Equipo equipo2	Inicializar un encuentro sin programar ni resultado y validar que los equipos sean diferentes.	IllegalArgumentException
22	getId	int	public	Encuentro		Consultar el identificador.	
23	getEquipo1	Equipo	public	Encuentro		Consultar el primer equipo.	
24	getEquipo2	Equipo	public	Encuentro		Consultar el segundo equipo.	
25	getMarcador1	Integer	public	Encuentro		Consultar el marcador del primer equipo.	
26	getMarcador2	Integer	public	Encuentro		Consultar el marcador del segundo equipo.	
27	getEstado	String	public	Encuentro		Consultar el estado del encuentro.	
28	getFecha	LocalDate	public	Encuentro		Consultar la fecha asignada.	
29	getHora	int	public	Encuentro		Consultar la hora asignada.	
30	getSede	String	public	Encuentro		Consultar la sede asignada.	
31	estaProgramado	boolean	public	Encuentro		Comprobar si tiene fecha asignada, incluso si ya finalizó.	
32	programar	void	public	Encuentro	LocalDate fecha, int hora, String sede	Validar la programación y asignarla si el encuentro no está finalizado.	IllegalArgumentException, IllegalStateException.
33	participaEquipo	boolean	public	Encuentro	Equipo equipo	Comprobar si el equipo es uno de los participantes.	

34	registrarResultado	void	public	Encuentro	int marcador1, int marcador2	Validar el resultado, calcular y sumar los puntos, guardar el marcador y finalizar el encuentro.	IllegalArgumentException, IllegalStateException.
35	validarMarcador	void	protected, abstract	Encuentro	int marcador1, int marcador2	Dejar la validación del marcador a cada deporte.	IllegalArgumentException
36	calcularPuntos	int[]	public, abstract	Encuentro	int marcador1, int marcador2	Dejar el cálculo de puntos a cada deporte.	IllegalArgumentException
37	getDeporte	String	public, abstract	Encuentro		Obtener el nombre del deporte definido por la subclase.	
38	getDescripcion	String	public	Encuentro		Describir el deporte, los equipos, el estado y el resultado cuando exista.	
39	FutbolSala	Constructor	public	FutbolSala	int id, Equipo equipo1, Equipo equipo2	Inicializar los datos heredados mediante super.	IllegalArgumentException
40	validarMarcador	void	protected	FutbolSala	int marcador1, int marcador2	Rechazar valores negativos y permitir empates.	IllegalArgumentException
41	calcularPuntos	int[]	public	FutbolSala	int marcador1, int marcador2	Validar el marcador y asignar 3 puntos por victoria, 1 por empate y 0 por derrota.	IllegalArgumentException
42	getDeporte	String	public	FutbolSala		Devolver “Fútbol sala”.	
43	Baloncesto	Constructor	public	Baloncesto	int id, Equipo equipo1, Equipo equipo2	Inicializar los datos heredados mediante super.	IllegalArgumentException
44	validarMarcador	void	protected	Baloncesto	int marcador1, int marcador2	Rechazar valores negativos y empates.	IllegalArgumentException
45	calcularPuntos	int[]	public	Baloncesto	int marcador1, int marcador2	Validar el marcador y asignar 2 puntos al ganador y 1 al perdedor.	IllegalArgumentException
46	getDeporte	String	public	Baloncesto		Devolver “Baloncesto”.	
47	Exhibicion	Constructor	public	Exhibicion	int id, String titulo	Validar los datos e inicializar una exhibición sin programar.	IllegalArgumentException
48	getId	int	public	Exhibicion		Consultar el identificador.	
49	getTitulo	String	public	Exhibicion		Consultar el título.	
50	getFecha	LocalDate	public	Exhibicion		Consultar la fecha asignada.	
51	getHora	int	public	Exhibicion		Consultar la hora asignada.	
52	getSede	String	public	Exhibicion		Consultar la sede asignada.	
53	getEstado	String	public	Exhibicion		Indicar si está sin programar o programada según su fecha.	
54	estaProgramado	boolean	public	Exhibicion		Comprobar si tiene fecha asignada.	
55	programar	void	public	Exhibicion	LocalDate fecha, int hora, String sede	Validar los datos y programar o reprogramar la exhibición.	IllegalArgumentException
56	participaEquipo	boolean	public	Exhibicion	Equipo equipo	Devolver false porque no tiene equipos.	
57	getDescripcion	String	public	Exhibicion		Obtener la descripción para el calendario.	
58	Copa	Constructor	public	Copa		Inicializar las listas y el siguiente identificador y cargar los datos iniciales.	
59	registrarEquipo	Equipo	public	Copa	String codigo, String nombre, String empresa	Validar los datos y la unicidad del código y registrar el equipo.	IllegalArgumentException
60	buscarEquipo	Equipo	public	Copa	String codigo	Buscar un equipo por su código.	IllegalArgumentException si no existe
61	crearFutbolSala	FutbolSala	public	Copa	String codigo1, String codigo2	Crear el encuentro y agregar sus referencias a las listas correspondientes.	IllegalArgumentException
62	crearBaloncesto	Baloncesto	public	Copa	String codigo1, String codigo2	Crear el encuentro y agregar sus referencias a las listas correspondientes.	IllegalArgumentException
63	crearExhibicion	Exhibicion	public	Copa	String titulo	Crear la exhibición y agregarla a los eventos.	IllegalArgumentException
64	buscarEvento	Programable	public	Copa	int id	Buscar un evento por su identificador.	IllegalArgumentException si no existe

65	buscarEncuentro	Puntuable	public	Copa	int id	Buscar un encuentro que pueda recibir resultados.	IllegalArgumentException si no existe
66	programarEvento	void	public	Copa	int id, LocalDate fecha, int hora, String sede	Comprobar conflictos y delegar la programación al evento.	IllegalArgumentException, IllegalStateException.
67	validarConflictos	void	private	Copa	Programable evento, LocalDate fecha, int hora, String sede	Detectar conflictos de sede o equipos, excluyendo al propio evento.	IllegalStateException
68	registrarResultado	void	public	Copa	int id, int marcador1, int marcador2	Buscar el encuentro y delegar el registro mediante Puntuable.	IllegalArgumentException, IllegalStateException.
69	getEquipos	List<Equipo>	public	Copa		Devolver una copia de la lista de equipos.	
70	getEventos	List<Programable>	public	Copa		Devolver una copia de la lista de eventos.	
71	getEncuentros	List<Puntuable>	public	Copa		Devolver una copia de la lista de encuentros.	
72	obtenerTablaPosiciones	List<Equipo>	public	Copa		Obtener una copia de los equipos ordenada mediante compareTo.	
73	iterator	Iterator<Programable>	public	Copa		Recorrer los eventos con programación en orden cronológico sin exponer la lista interna.	
74	cargarDatosIniciales	void	private	Copa		Cargar los equipos, eventos y resultados iniciales mediante las operaciones del modelo. Se invoca desde el constructor.	IllegalArgumentException, IllegalStateException si los datos son inválidos.
75	Vista	Constructor	public	Vista		Inicializar el lector de consola.	
76	mostrarMenu	int	public	Vista		Presentar las opciones del programa y devolver la opción elegida.	Maneja NumberFormatException
77	leerTexto	String	public	Vista	String mensaje	Solicitar y leer texto.	
78	leerEntero	int	public	Vista	String mensaje	Leer un entero y repetir la solicitud si el formato es incorrecto.	Maneja NumberFormatException
79	leerFecha	LocalDate	public	Vista	String mensaje	Leer una fecha y repetir la solicitud si el formato es incorrecto.	Maneja DateTimeParseException
80	mostrarMensaje	void	public	Vista	String mensaje	Mostrar confirmaciones o errores.	
81	mostrarEquipos	void	public	Vista	List<Equipo> equipos	Presentar los equipos registrados.	
82	mostrarEventos	void	public	Vista	List<Programable> eventos	Presentar los eventos disponibles para seleccionar.	
83	mostrarEncuentros	void	public	Vista	List<Puntuable> encuentros	Presentar los encuentros disponibles para seleccionar.	
84	mostrarTabla	void	public	Vista	List<Equipo> equipos	Mostrar los equipos y sus puntos en el orden recibido.	
85	mostrarCalendario	void	public	Vista	Iterable<Programable> calendario	Recorrer y mostrar los eventos del calendario.	
86	Controlador	Constructor	public	Controlador		Crear la Copa y la Vista que usará el controlador.	
87	iniciar	void	public	Controlador		Mantener activo el menú y coordinar las operaciones hasta que el usuario decida salir.	Maneja IllegalArgumentException, IllegalStateException.
88	gestionarRegistroEquipo	void	private	Controlador		Solicitar código, nombre y empresa y delegar el registro a la copa.	IllegalArgumentException
89	gestionarCreacionEncuentro	void	private	Controlador		Solicitar el deporte y los equipos y delegar la creación a la copa.	IllegalArgumentException
90	gestionarCreacionExhibicion	void	private	Controlador		Solicitar el título y delegar la creación a la copa.	IllegalArgumentException
91	gestionarProgramacion	void	private	Controlador		Solicitar el evento, la fecha, la hora y la sede para programar o reprogramar.	IllegalArgumentException, IllegalStateException

92	gestionarResultado	void	private	Controlador		Solicitar el encuentro y los marcadores y delegar el registro.	IllegalArgumentException, IllegalStateException
93	gestionarTabla	void	private	Controlador		Obtener la tabla ordenada de la copa y enviarla a la vista.	
94	gestionarCalendario	void	private	Controlador		Enviar la copa a la vista para que recorra su calendario.	
95	main	void	public static	Main	String[] args	Crear el Controlador e iniciarlo. La copa y la carga de datos se crean dentro del controlador.	